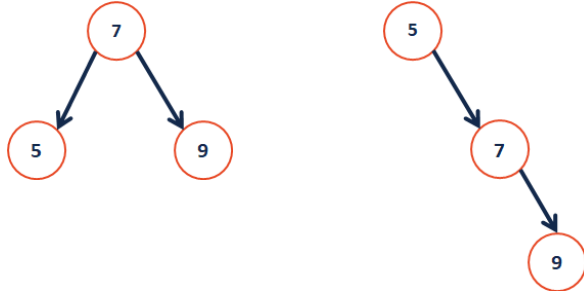


Height Balance on BST

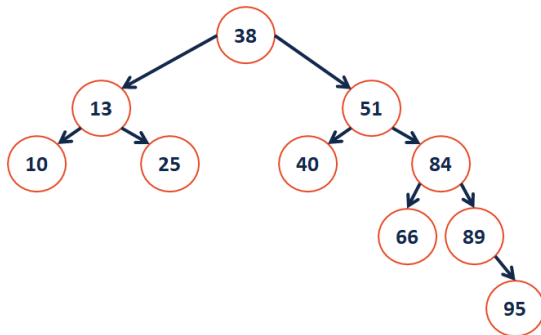
What tree makes you happier?



We define the **height balance** (b) of a BST to be:

We define a BST tree T to be **height balanced** if

What is the lowest node that is out of balance?



BST Rotations

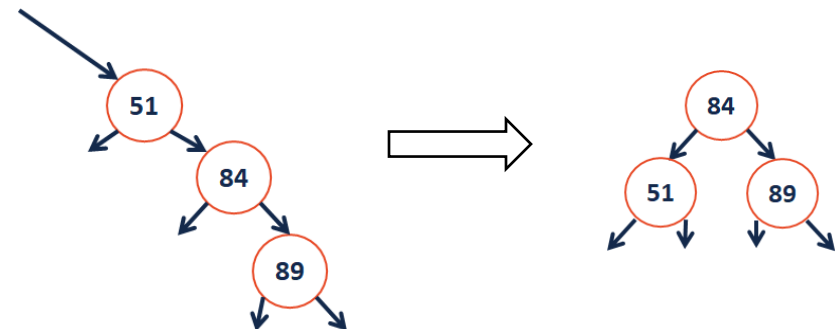
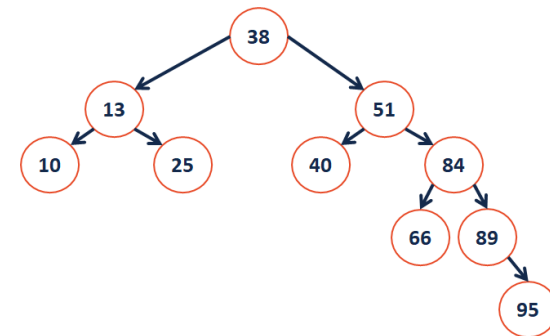
Every BST rotation will maintain two properties:

1.

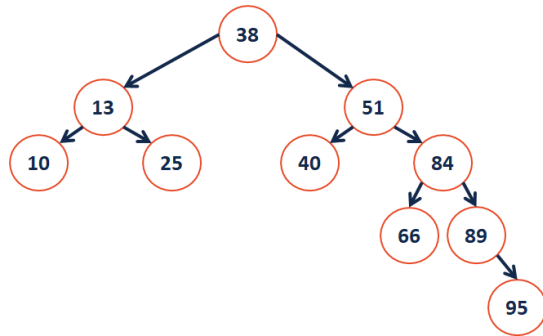
2.

Example: Defining a Rotation

Given a _____ rotation about 51, we can label 4 subtrees:



Implementing a left rotation:



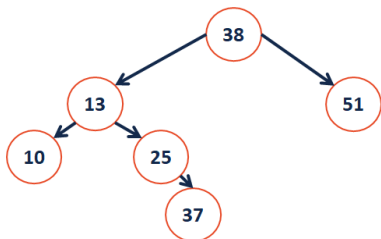
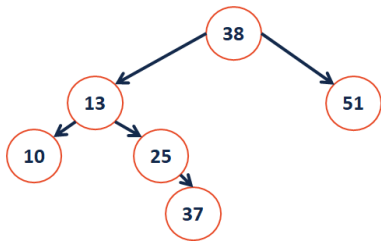
BST Rotation Summary:

1. Four kinds of rotations (L, R, LR, and RL)
2. All rotations are local
3. All rotations run in constant time, $O(1)$
4. BST property is maintained!

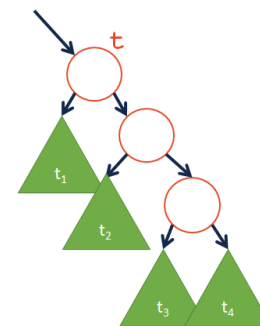
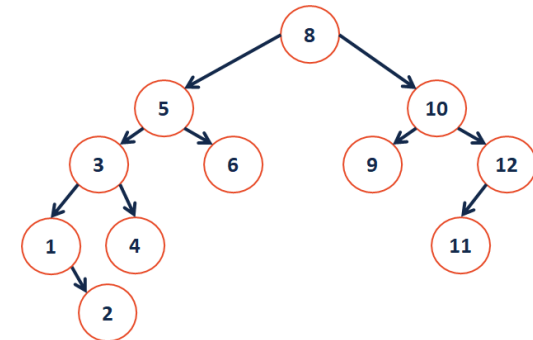
Overall Goal:

...and we call these trees:

Performing a right rotation:

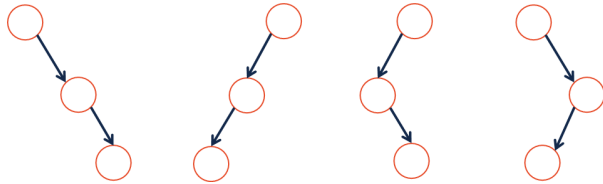


AVL Trees:



Theorem: If an insertion occurred in subtrees t_3 or t_4 and a subtree was detected at t , then a _____ rotation about t restores the balance of the tree.

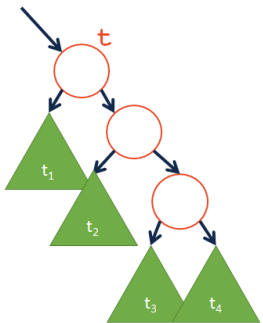
Four AVL Rotation Templates:



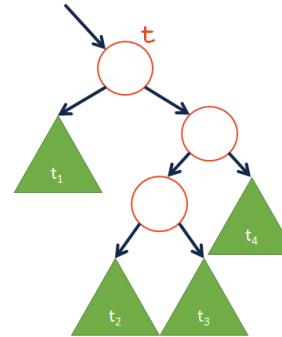
Detecting Imbalance:

b =

Theorem #1: If an insertion occurred in subtrees t_3 or t_4 and a subtree was detected at t , then a _____ rotation about t restores the balance of the tree.

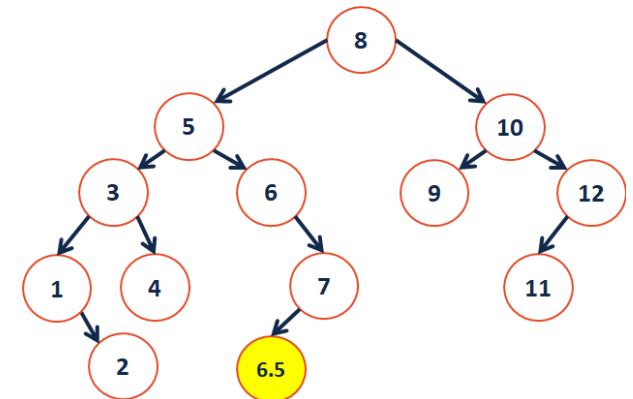


Theorem #2: If an insertion occurred in subtrees t_2 or t_3 and a subtree was detected at t , then a _____ rotation about t restores the balance of the tree.



AVL Insertion

Pseudocode:



AVL.h (snippet)

```
1 struct TreeNode {
2     T key;
3     unsigned height;
4     TreeNode *left;
5     TreeNode *right;
6 }
```

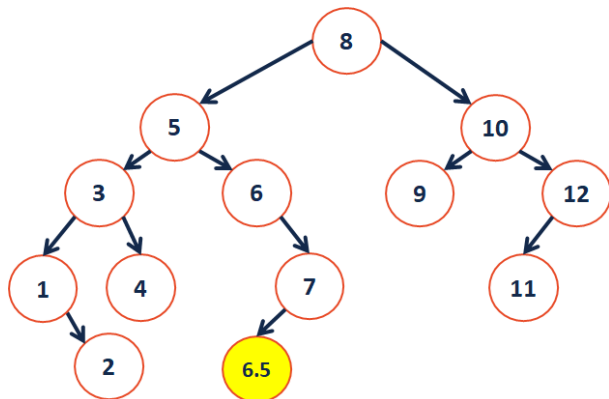
AVL Insertion

AVL.cpp (snippet)

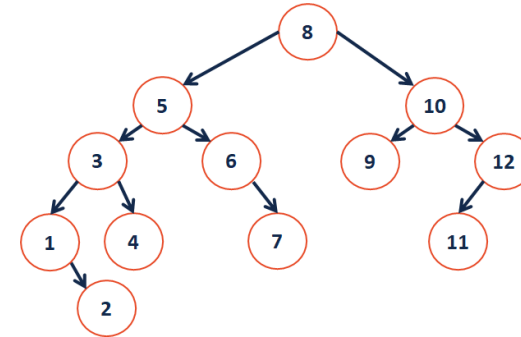
```

1  template <class T>
2  void AVLTree<T>::_insert(const T & x, treeNode<T> * & t ) {
3      if( t == NULL ) {
4          t = new TreeNode<T>(x, 0, NULL, NULL);
5      }
6      else if( x < t->key ) {
7          _insert( x, t->left );
8          int balance = height(t->right) - height(t->left);
9          int leftBalance = height(t->left->right)
10             - height(t->left->left);
11          if ( balance == -2 ) {
12              if ( leftBalance == -1 ) { rotate_____ ( t ); }
13              else { rotate_____ ( t ); }
14          }
15      }
16      else if( x > t->key ) {
17          _insert( x, t->right );
18          int balance = height(t->right) - height(t->left);
19          int rightBalance = height(t->right->right)
20             - height(t->right->left);
21          if( balance == 2 ) {
22              if( rightBalance == 1 ) { rotate_____ ( t ); }
23              else { rotate_____ ( t ); }
24          }
25      }
26      t->height = 1 + max(height(t->left), height(t->right));
27  }

```



AVL Removal



AVL Analysis

We know: _____. We will argue: $h =$ _____.

Iterators in MP4

1. Iterators give client code access to our data and
2. Encapsulate access to our data regardless of data structure
...and iterators are not new – we provided one for you in MP3!

As part of MP4, you will write your own iterator:



FloodFillImage.cpp (partial)

```

ImageTraversal & traversal = /* ... */;
for (const Point & p : traversal) {
}

```

Today's Goal:**Motivation:**

Big-O is defined as:

Visually:

**Plan of Action:**

We will begin by defining a function that defines the least number of nodes in an AVL tree of height h .

$N(h)$:

State a Theorem:

An AVL tree of height h has at least _____.

I. Consider an AVL tree and let h denote its height.

II. Case: _____

III. Case: _____

IV. Case: _____

By an inductive hypothesis (IH):

We show that:

V. Using a proof by induction, we have shown that:

...and by inverting our finding:

Summary of Balanced BSTs:

Advantages	Disadvantages

CS 225 – Things To Be Doing:

- 1. MP4 extra credit submission is next Monday!
- 2. lab_huffman is due this Sunday.